

Rethinking Attack Path Management: A New Metric for Choke Points in Attack Graphs

Yumeng Zhang

The University of Adelaide

Adelaide, Australia

yumeng.zhang01@adelaide.edu.au

Max Ward

University of Western Australia

Perth, Australia

max.ward@uwa.edu.au

Hung Nguyen

The University of Adelaide

Adelaide, Australia

hung.nguyen@adelaide.edu.au

Abstract—In this paper, we propose a novel choke point metric for the elimination of attack paths. Our study is motivated by its applications in the widely used Active Directory (AD) attack graphs. Choke points are typically defined as critical locations where the largest number of attack paths converge. Identifying these choke points is crucial. Enumeration of all attack paths is implied in this definition, but the immensity of paths in AD attack graphs makes the task extremely challenging. Consequently, industry solutions and research often rely on mapping only the shortest paths or prioritizing their elimination as a method for hardening AD attack graphs. We theoretically describe and empirically measure major limitations with the shortest path approach. To address the limitations, we introduce a new choke point metric that quantifies the intersection of connections rather than attack paths, which improves upon shortest path mapping. Additionally, we present human experiments to observe how white-hat hackers use shortest path mapping, provide simple graph examples that visually demonstrate failure cases where shortest path-based methods do not yield optimal results, and conduct experiments with a diverse set of real-world and synthetic AD datasets. From the results, we conclude that uninformed attack path mapping cannot capture the complexity of the attack path composition in a real-world attack, and reliance on shortest path mapping leads to significant volatility in security-hardening outcomes. In contrast, the connection-based choke point metric we propose offers greater optimality and utility in mitigating the attack surface.

Index Terms—Active Directory attack graph, attack path, choke points

I. INTRODUCTION

Cyber attack graphs model the chain of events (conceptual or physical) that lead to successful cyber attacks. The term “attack graph” was first introduced around 1998 [48], [55], [39], and since then there have been more than 90 different definitions of attack graphs and methods to construct them, see for example [49], [56], [41], [3], [27]. One of the most prominent and actively used attack graph models is the Active Directory (AD) attack graph that models identity-based snowball attacks in Windows systems, first proposed in [9], developed further in [5] and then commercialized by the BloodHound tool [52].

An AD attack path represents a sequence of access privileges exploits that enable the attacker to hop between machines and user accounts. Figure 1 shows an example of such an attack path. Attackers typically use these attack graphs to navigate their way from a low-privilege account to high-value targets such as the DomainAdmins (DA) group—giving them

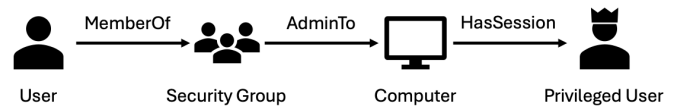


Fig. 1. An example of an AD attack path.

persistence. A typical AD attack graph contains millions, if not billions, of attack paths exacerbating the exposure of high-value assets within the network [24]. Being able to audit, identify, and eliminate attack paths is crucial for containing and mitigating the impact of security breaches within an organization [50]. There exist numerous Microsoft resources providing guidance on reducing attack paths to the greatest extent possible [32], [30], [31], [33], [29]. An important approach is through the identification of *choke points*, which in [32] are defined as locations where multiple attack paths intersect on their way to high-value assets.

There have been numerous implementations of security-hardening concepts similar to choke points. As an example, GoodHound [34] is a popular open-source AD security-monitoring tool that reports attack paths as well as finding the weakest links evaluated based on the number of enumerated shortest paths that share the edge. Another open-source AD attack path managing tool called PlumHound [18] also provides algorithms for identifying weakest links and busiest paths, both of which rely on the enumeration of shortest paths. Numerous papers in academia approached it as the shortest path interdiction problem, which aims to find a limited set of edges removing which would eliminate shortest paths to the greatest extent possible [14], [15], [12], [61]. On the other hand, commercial products such as BloodHound Enterprise [50] identify choke points as misconfigurations closest to the high-value target enabling a large percentage of users and computers with a path to the target. Note that the choke point definition given by BloodHound Enterprise does not explicitly state “shortest paths”, which implies that *any* attack path enabling a connection with high-value targets might be taken into consideration.

There are several limitations of the current approaches to attack path elimination in AD. First, counting the total number of simple paths in a directed graph is in fact intractable as it is a #P-complete problem [53]. Finding a solution for an

accurate and comprehensive evaluation of choke points based on attack path mapping and counting is therefore extremely challenging. Second, the abovementioned products heavily rely on the enumeration of shortest paths, which is evidently a highly simplified approximation for the quantification and visualization of the connectivity. Not only does it ignore paths of longer lengths, but it even disregards the presence of paths of the same shortest length. In one of our experiments where the goal is to eliminate all paths of the shortest length, we observed that up to 79.71% of paths can be omitted from elimination because of this arbitrarily approximated counting of shortest paths. *These issues stem from the lack of an adequate metric for choke points that could efficiently and objectively guide the elimination of paths from low-value assets to high-value targets.*

In this work, we propose a new choke point metric that eliminates the arbitrariness inherent in the shortest path-based choke point metric, bypasses the extreme challenges posed by the simple path counting problem, and transitions the defense effort into a new direction that extends beyond mere attack path elimination, thereby improving the utility of the choke point metric. To accomplish this goal, we refine the definition of the attack surface to include not only the number of attack paths a high-value target is exposed to but also the number of potentially compromised network entities with a path to the target. The choke point metric we propose under this revised definition of the attack surface now emphasizes the minimization of *connections* in addition to eliminating attack paths. That is, we focus on cutting the high-value target away from the largest attack surface possible. Our definition is similar to the sparsest cut problem studied in [6], [9] for finding network bottlenecks, and is related to the concept of *bridges* in graph theory explored in [57], [17]. These metrics, however, are more suited for reducing the overall network connectivity, whereas our security goal is to reduce the connectivity between a predefined set of nodes from a target node so as to cut the high-value target from the potential attack entries.

Overall, this work presents a novel definition of attack path choke points, initially investigated algorithmically in [62]. In this paper, we comprehensively contrast it against the existing body of work that utilizes shortest paths and show that our new framework for attack graph analysis is more effective. Our contributions are:

- Refined definition of the attack surface and a novel choke point metric for less approximation and better metric utility compared with attack path and shortest path mapping-based methods.
- Clear and concise graph examples visually demonstrate the limitations of using shortest path mapping alone.
- A human study from which we observed how shortest path mapping is adopted by white-hat hackers in real-world AD attacks.
- Experiments with both real-world and synthetic AD attack graphs to strengthen our analysis and arguments. The real-world data includes a dataset collected from

a university AD network, as well as authentication event data published by Los Alamos National Laboratory (LANL). Additionally, various AD simulation packages are employed to generate synthetic AD attack graphs for a more comprehensive evaluation across diverse datasets and scenarios.

The paper is organized as follows: we first provide a review of attack graphs, security-hardening solutions and choke points in Section II. This is then followed by definitions of existing choke point metrics to be analyzed and illustrations of break cases that necessitate the establishment of the new metric. Finally in Section IV, we describe the think-aloud study conducted with white-hat hackers, followed by the experimental results comparing all choke point metrics defined in this paper using a large suite of AD datasets to show the better performance the new metric provides.

II. BACKGROUND

A. Attack graphs

Various types of attack graphs exist, each capturing network composition and vulnerabilities in distinct ways to support the development of different security and defense measures.

In [21], the attack graph of a system is modeled as a finite state machine, based on which the authors studied the problem of removing a minimal number of atomic attacks such that the success state cannot be reached from an initial state. In the face of the scalability issues associated with the state enumeration attack graphs, the exploit-dependency graph became a prominent form of attack graph [59]. It is a logic-based attack graph that models conditions and exploits in an attack as nodes while edges connect a condition and an exploit. MulVAL [43], [42] is an efficient tool for generating exploit-dependency attack graphs using logic programming. The attack graph generation in MulVAL is efficient and scalable. It is suited for analyzing multi-stage, multi-host attacks. Other examples using logic-based attack graphs can be found in [8], [10], [60], [54], [4]. These attack graph models are used for developing various security-hardening measures such as optimizing the allocation of attack countermeasures [8], [10], [60], diversifying network services [4] and identifying critical attack assets in the network [54].

AD attack graph is a more recent type of attack graph designed specifically for modeling identity-based snowball attacks in AD networks. It is a directed graph in which nodes model network entities such as user accounts, computers, security groups, organizational units (OUs), Group Policy Objects (GPOs) and Domains. Directed edges represent the access privileges associated with each node, including AdminTo, GenericWrite, Owns, and CanRemoteDesktop. Overall, AD attack graphs focus on showing entities in an identity network and their respective access privileges, and how those privileges enable lateral movements between machines and user accounts, establishing connections between high-value assets and low-privileged components that may be overlooked. All user accounts and computers modeled in the AD attack graph,

which encompasses a significant portion of the nodes, could serve as initial entry points in an attack, forming the attack surface that should be considered in risk mitigation efforts.

B. Security hardening in AD attack graphs

In AD attack graphs, shortest path mapping remains a crucial method for visualizing and guiding the mitigation of the attack surface. Here, shortest paths refer to paths with the least number of edges between a specific pair of nodes. Notable examples using this method can be found in tools like BloodHound, GoodHound and PlumHound. BloodHound [52], one of the most widely used AD attack path mapping tools in the industry, has been referred to in a report by the Cybersecurity and Infrastructure Security Agency (CISA) as a tool leveraged by attackers in successful cyber intrusions on AD systems [7]. Additionally, it is recommended by the U.S. Department of Homeland Security for defensive purposes [1]. Inspired by BloodHound, GoodHound [34] was developed and became another popular open-source AD security monitoring and reporting tool. In GoodHound, the attack path mapping is also limited to the shortest paths. PlumHound [18] is another attack path reporting tool based on BloodHound. In its path analyzing modules for identifying weakest links and busiest paths, shortest paths retain an important role.

Motivated by the importance of shortest paths in successful attacks, numerous research papers on hardening AD attack graphs defined the problem as the shortest path interdiction problem [14], [15], [12], [61]. The problem is approached with a Stackelberg game model, in which the attacker's goal is to reach DA through a shortest attack path, while the defender's objective is to minimize the attacker's success probability by removing edges under a budget. All of the papers assumed the attacker invariably chooses optimal attack paths, which are the shortest paths in which edges either have unit edge weights or values quantifying the exploitation difficulty. One key advancement in this problem formulation is that the shortest paths here do not exclusively refer to those of the absolute shortest length. If within the defense budget, paths of the second or third shortest lengths might also be eliminated. Nevertheless, at the core of this problem formulation, the elimination of shortest paths is given the highest priority in the defense effort.

In comparison, our goal in this work is to propose a new direction that optimizes the elimination of general attack paths and reduces the assumptions imposed on what is optimal for the attacker. The shortest paths are not necessarily optimal in real-world attacks due to factors such as the attacker's skill level and risk tolerance [26], as well as the approximate nature of the attack graph, where some edges may not exist. We study the security-hardening problem in an unweighted AD attack graph—the situation in which the attacker's success probability associated with exploiting each network entity or access privilege is not available. Since these probabilistic parameters are challenging to obtain in practice, we endeavor to mitigate the security risk by reducing the level of network connectivity the high-value asset is exposed to.

C. Choke points

In this work, the security-hardening method we study is the identification of choke points in AD attack graphs for the elimination of general attack paths. This approach shares a common objective with the methods reviewed above: minimizing defense efforts while maximizing overall security benefits.

The notion of a choke point has a longstanding presence within multiple fields. Choke points are often vaguely defined as “narrow passageways connecting one region to another” in the maritime and topographical context [2], [47]. They can also be locations where most traffic passes through and congestion is likely to happen in transportation networks [28]. In computer networks, a choke point can be a network bottleneck in terms of bandwidth that is of deterministic importance to seamless packet transmission [44].

Below, we summarize choke points and related concepts in the context of graphs. Similar ideas are referred to in the literature as “bottlenecks”. Topological concepts adopted for describing choke points include the node degree [40], betweenness centrality [16], and the number of paths passing through them [10], [38], [11], [32]. A more detailed explanation of choke points in AD attack graphs describes them as locations where most attack paths converge before reaching the high-value target, enabling a large percentage of users and computers with a path, therefore eliminating them brings the greatest attack path elimination benefit for securing the high-value target [45], [13], [51], [50]. However, given the immense scale of an AD attack graph, developing a practical algorithm for accurate attack path counting and mapping is extremely challenging. The problem of counting simple paths in a graph has in fact been proven to be #P-complete [53]. It is intractable.

In addition to this general path-based interpretation, there are also perspectives and implementations that only consider shortest paths between relevant node pairs in the evaluation of choke points. Examples can be found in [34], [18], [19], [20]. Due to the prominence of BloodHound in cyber attacks targeting AD, many AD defensive solutions [34], [46], [18] are built based on BloodHound relying on Neo4j graph database and its Shortest Path cypher query for the mapping of attack paths. Note that the Shortest Path query returns only one (could be arbitrary) shortest path among all shortest paths between a pair of nodes, and is therefore only a highly simplified approximation method to substitute the enumeration of all attack paths.

In this work, we refine the definition of the attack surface to include not only the number of attack paths a high-value target is exposed to but also the number of potential attack entries that have a path to the high-value target. The corresponding choke point metric now includes the minimization of the number of vulnerable network entities connected to the high-value target in addition to the elimination of attack paths. This problem was first investigated algorithmically in [62], where proof showing that it is NP-hard and an anytime algorithm are provided. Since it is possible to develop algorithms efficient

enough with a good level of optimality, the new choke point metric definition is capable of bypassing the extreme challenges posed by the simple path counting problem and eliminating the arbitrariness of using the shortest path mapping as a replacement. Furthermore, it helps transition the defense effort into reducing the number of vulnerable network entities connected to the high-value target, which extends beyond mere attack path elimination. Despite the algorithmic study completed in [62], an investigation into its security implication as a novel choke point metric compared to the traditional attack path mapping-based approach remains a research gap.

Finding bottlenecks in complex networks with a goal similar to ours can be found in [6], where the problem is modeled as the minimum conductance problem, which is equivalent to finding the sparsest cut. Similarly in [9], the sparsest cut problem is studied to find small sets of high-impact configuration changes. The sparsest cut problem measures how well connected different subgraphs are in the network relative to the subgraphs' internal connectivity. Another related concept in graph theory is called the bridge [17], [57], which can be used to evaluate the extent to which a node or an edge connects different regions in a graph, therefore can be used for identifying potential bottlenecks in the information flow. These methods are more suited for studying graph clustering, increasing the number of connected components and reducing the overall network connectivity. They do not provide a focus on reducing the connectivity between the attack surface and the high-value target node, which is our security goal. The security metrics we study in the paper are different approaches from these existing graph concepts.

To summarize, in the background where attack path choke points in AD attack graphs are evaluated based on the number of simple paths, numerous existing solutions revolve around the mapping and elimination of shortest paths, either as an alternative to the #P-complete simple path counting problem or due to assumptions about optimal attacker's strategies. In our work, we propose a new choke point metric to address the limitations of the attack path mapping-based metrics. Like previous solutions, it aims to decrease the connectivity exposing the high-value target to reduce the attack surface. However, our metric offers improved accuracy and greater utility, therefore moving beyond the approximations of past approaches.

III. METRICS AND ALGORITHMS

A. Attack graph model

We define an AD attack graph to be a directed graph $G = (V, E)$, in which a set of vertices V model all physical and virtual network components. In AD networks, they correspond to components such as user accounts, computers, security groups, organizational units (OUs) and domains. The set of directed edges E represents exploitable relationships or privileges that enable the attacker to make lateral movements. We assume that the high-value target that the attacker aims for is DA, and we use t to denote this ultimate target. Lastly,

we use $S \subseteq V$ to denote the attack surface that we strive to mitigate.

In this paper, we assume that no entity is entirely secure; all user accounts and computers within an enterprise network that an attacker could potentially exploit constitute the attack surface. Despite the abundant varieties of network objects modeled in the AD attack graph, it is the user accounts and computers that serve as the actual foothold for the attacker which they can directly manipulate to make lateral movements. If there exists empirical evidence showing certain network entities are more vulnerable or have already been compromised, S can be easily adjusted accordingly to better suit the specific use case without affecting this problem definition and the corresponding algorithmic solutions.

Definition 1 (Attack Path). An attack path P is defined as a sequence of vertices $P = \langle v_1, \dots, v_n \rangle$, in which $v_1 \in S$, $v_n = t$, $v_i \in V$ and each $(v_i, v_{i+1}) \in E$ are the edges in the path, $1 \leq i < n$. We assume AD attack paths to be simple paths—paths with no cycles so that each node in a path is unique. Formally, $v_i \neq v_j$ for any pair of vertices in the path $v_i, v_j \in P$. An attack path shows the network components and associated access privileges that an attacker exploits during a lateral movement.

Definition 2 (Path Length). We define the edges to have unit edge weights, and the length of a path $\text{len}(P)$ corresponds to the number of edges on a path, namely $\text{len}(P) = n - 1$ where $P = \langle v_1, \dots, v_n \rangle$. It quantifies the number of access privileges exploited by the attacker along an attack path.

Definition 3 (Shortest Path). The shortest path from a node u to node v , denoted as $SP(u, v)$, is a path between u and v such that the length of the path $\text{len}(SP(u, v)) \leq \text{len}(P)$ for any path $P = \langle u, \dots, v \rangle$. A shortest path represents the minimum number of exploitations required to reach a specific target, typically assumed to reflect the least effort necessary to complete a mission.

B. Attack path-based metrics

In this section, we formalize the definition of choke points when the attack surface is defined solely in terms of the number of attack paths DA is exposed to. In line with this, an accurate definition of the choke point would be the location where the greatest number of attack paths converge. This requires an exact counting of all simple paths from S to t . A formal definition of this choke point metric is given as Definition 4.

Definition 4 (Attack path-based narrowness). For an attack graph $G = (V, E)$ with source nodes S and a target node t , we define AP to be a set of all attack paths between each $s \in S$ and t . The narrowness of an edge $(u, v) \in E$ under this metric is evaluated as

$$ch(u, v) = \sum_{P \in AP} I_{(u, v) \in P}$$

where $I_{(u, v) \in P}$ is the indicator function, $I_{(u, v) \in P} = 1$ if edge (u, v) is in path P and 0 otherwise.

The word “narrowness” here is borrowed from maritime studies, where choke points are channels in global maritime transport that most traffic goes through. Narrower channels are more prone to congestion, which would cause delays and major disruptions in global trade, therefore would be considered “chokier”. In the context of AD attack graphs, “narrowness” can be mapped to that in the maritime context by considering edges as channels with a major impact on the connectivity between the attack surface and the high-value target. In later sections, we will be considering a set of edges together being the choke point. Among different sets of edges with the same level of impact on the connectivity, the set with fewer edges would be considered narrower.

The problem with metric ch is that it is impractical to enumerate all attack paths, and the problem of counting simple paths is #P-complete [53]. As mentioned before, all existing methods resort to mapping and counting the shortest paths, in which only one arbitrary shortest path is found from each source node to the target node. We formalize this metric as Definition 5.

Definition 5 (Shortest path-based Narrowness). Given $G = (V, E)$, source nodes S and a target node t . We use SP to denote the set of shortest paths from each source node $s \in S$ to the target node t , one path for each s . The shortest path-based narrowness of an edge (u, v) is thus calculated as

$$chSP(u, v) = \sum_{P \in SP} I_{(u, v) \in P}$$

A pseudocode for computing $chSP$ is provided in Algorithm 2 in the Appendix.

The extent of approximation from ch to $chSP$ is alarmingly excessive. $chSP$ even disregards the presence of multiple other paths with the same shortest length. In cases of ties, the chosen shortest path is arbitrary. An improvement we can make over $chSP$ would be to identify *all* shortest paths for the choke point evaluation. As more attack paths are mapped out, the complexity of the represented connectivity between nodes would be improved. To distinguish this from $chSP$, we extract this approach as a separate metric.

Definition 6 (All Shortest Paths-based narrowness). Given $G = (V, E)$, source nodes S and a target node t , we define ASP to be the set of all shortest paths from each source node $s \in S$ to the target node t . Note that the shortest path length from each source node could be different. The shortest path-based narrowness of an edge (u, v) is calculated as

$$chASP(u, v) = \sum_{P \in ASP} I_{(u, v) \in P}$$

Proposition 1. We use $G^* = (V^*, E^*)$ to denote the directed graph constructed with ASP . G^* is an acyclic graph, and only contains paths in ASP from each $s \in S$ to t . That is, no unintentional paths would be created as we build G^* with nodes and edges in ASP .

Proof for Proposition 1 is provided in the Appendix.

The computation of $chASP$ is polynomial when G^* is acyclic. We present in Algorithm 1 a Dynamic Programming (DP) algorithm for computing the number of simple paths passing through each edge in a directed acyclic graph. As stated in Proposition 1, G^* only contains paths in ASP , the number of simple paths passing through each edge in G^* would therefore be equivalent to the number of all shortest paths passing through the edge in G .

In Algorithm 1, function $DP()$ computes the number of paths from a node u to x in the input graph G^* . Calls to the function are memorized. To compute $chASP$, we first use $BUILD_GRAPH$ to build G^* that consists only of ASP , then we go through each edge $(u, v) \in E^*$ and use the DP algorithm to calculate the number of paths from s to u and from v to t . The number of paths passing edge (u, v) can then be calculated by multiplying the two values. If there are multiple source nodes, they can be converted to one by adding one super-source node that has one edge entering each source node, and then taking the super-source as the single source node. Overall, the computational complexity of this algorithm is $O(|V| + |E|)$.

The number of attack paths that could be missed from elimination when using $chSP$ and $chASP$ is striking. To provide a preview, we present a brief summary of the experiment results here. Detailed explanations of the experiments are provided later in Section IV-C. In our first experiment where the security goal is to eliminate ASP from G , we observed that $chSP$ performs poorly, omitting up to 79.71% of the paths from elimination compared to $chASP$ across various AD datasets. After we adjusted the security goal to eliminating 2000 paths which are not necessarily the shortest, choke points evaluated with ch should give the optimal result as this metric considers the full space of the attack paths. $chSP$ and $chASP$, on the other hand, now both consider only a partial attack path space, and the elimination performance suffers as a result—as large as 1276 projected attack paths can be omitted from elimination by $chASP$, and 1186 for $chSP$. Since both $chSP$ and $chASP$ are now an arbitrary approximation of ch , neither of them has a guaranteed advantage over the other. As discussed previously, counting all simple paths in a directed graph is intractable, therefore this approximation and the resulting arbitrariness in the metric’s performance are likely to be unavoidable.

C. A new choke point metric—connection-based choke point metric

In this section, we define the new attack path choke point metric for reducing the connectivity that DA is exposed to. Recall that the definition of the attack surface now includes not only the number of attack paths but also the number of potentially compromised network entities with a path to DA. Accordingly, the choke point is redefined as the intersection of connections, extending beyond mere attack paths.

Given a source node $s \in S$ and a target node t , as long as there exists one (s, t) -path, there is a connection. One (s, t) -path is inadequate to represent the connection when the goal is to disconnect s from t , namely eliminating all (s, t) -paths.

Algorithm 1 Algorithm for computing *chASP*

```
1: function chASP( $G, S, t$ )
2:    $\text{edge\_stats} \leftarrow \{\}$ 
3:    $\text{count\_from\_s} \leftarrow \{\}$ 
4:    $\text{count\_to\_t} \leftarrow \{\}$ 
5:    $G^* \leftarrow \text{BUILD\_GRAPH}(G, S, t)$ 
6:    $G^{*T} \leftarrow \text{REVERSE}(G^*) \triangleright G^{*T}$  is a transpose of  $G^*$ 
7:   for each edge  $(u, v)$  in  $E^*$  do
8:      $s\_count \leftarrow \text{DP}(G^{*T}, \text{count\_from\_s}, u, s)$ 
9:      $t\_count \leftarrow \text{DP}(G^*, \text{count\_to\_t}, v, t)$ 
10:     $\text{edge\_stats}[(u, v)] \leftarrow s\_count \times t\_count$ 
11:   return  $\text{edge\_stats}$ 
12: function BUILD_GRAPH( $G, S, t$ )
13:    $V^* \leftarrow \emptyset, E^* \leftarrow \emptyset, G^* \leftarrow (V^*, E^*)$ 
14:    $\text{dist} \leftarrow \{\}$ 
15:    $\text{next\_nodes} \leftarrow \{\}$ 
16:    $\text{pq} \leftarrow$  a priority queue of nodes, ranked in increasing
   order of the shortest path length from the node to  $t$ 
17:    $\text{pq.add}(t, 0)$ 
18:    $\text{dist}[t] \leftarrow 0$ 
19:   while  $\text{len}(\text{pq}) > 0$  do
20:      $v, d = \text{pq.popitem}() \triangleright$  top node and its distance
21:     if  $d > \text{dist}[v]$  then
22:       continue
23:     for each predecessor  $u$  of  $v$  in  $G$  do
24:       if  $u$  not in  $\text{dist}$  or  $d + 1 < \text{dist}[u]$  then
25:          $\text{pq.add}(u, d + 1)$ 
26:          $\text{dist}[u] = d + 1$ 
27:          $\text{next\_nodes}[u] = \{v\}$ 
28:       else if  $d + 1 = \text{dist}[u]$  then
29:          $\text{next\_nodes}[u].\text{add}(v)$ 
30:   for each node  $u$  in  $\text{next\_nodes.keys}$  do
31:      $V^* \leftarrow V^* \cup \{u\}$ 
32:     for each node  $v$  in  $\text{next\_nodes}[u]$  do
33:        $E^* \leftarrow E^* \cup \{(u, v)\}$ 
34:   return  $G^*$ 
35: function DP( $G^*, \text{memo}, u, x$ )
36:   if  $u$  in  $\text{memo}$  then
37:     return  $\text{memo}[u]$ 
38:   if  $u = x$  then
39:     return 1
40:    $\text{total\_paths} \leftarrow 0$ 
41:   for each successor  $v$  of  $u$  in  $G^*$  do
42:      $\text{total\_paths} \leftarrow \text{total\_paths} + \text{DP}(G^*, \text{memo}, v, x)$ 
43:    $\text{memo}[u] \leftarrow \text{total\_paths}$ 
44:   return  $\text{memo}[u]$ 
```

The presence of thousands if not millions of other (s, t) -paths must not be forgotten.

The *minimum cut* problem is a classic graph theory problem and is polynomial time solvable [25]. Given two nodes s and t in a directed graph, if all edges have the capacity of 1, solving the minimum cut problem in this graph would give

the minimum number of edges to remove that would partition the graph into two disconnected subgraphs, where s and t are in different subgraphs. This not only ensures that no path remains between s and t , but it also optimizes the number of edges we remove, therefore ensuring their “narrowness”.

In the minimum cut problem, it is possible for the size of a minimum cut between a single source and sink to be the same as that for multiple sources. That is, suppose the size of the minimum cut is a fixed value, the number of source nodes can be adjusted while meeting the constraint on the size of the minimum cut. Therefore, a choke point could be a minimum cut that disconnects the largest number of source nodes.

Formally, we define our new metric as follows.

Definition 7 (Connection-based narrowness). Given the directed graph $G = (V, E)$, a set of source nodes S and a target node t , the narrowness of an edge is evaluated as

$$\text{chConn}(u, v) = \sum_{s \in S} I_{E-(u, v)}(s, t)$$

where the indicator function $I_{E-(u, v)}(s, t) = 1$ if there exists at least one path from s to t when edge (u, v) is removed, and 0 otherwise.

We use b to denote the number of edges evaluated to be removed simultaneously as choke points. It can also be considered as the defender’s budget limiting the number of edges that can be removed. Usually, b would be 1 to represent one narrowest location enabling the greatest number of connections, and this problem is polynomial time solvable. We can compute the number of connected source nodes by a depth-first search (DFS). Each edge can be evaluated individually by removing that edge and running DFS. All edges can then be ranked based on the disconnected sources. A pseudocode is provided in Algorithm 3 in the Appendix.

If we allow $b > 1$, the problem becomes an NP-hard combinatorial optimization problem. The proof is provided in [62]. In the same paper, a practical anytime algorithm has been provided. For this paper, we only adopt the Integer Linear Programming solution from [62] to obtain results with exact optimality for comparisons in later sections.

D. Breaking cases for attack path-based metrics

It is challenging to estimate the extent to which the attack path elimination result suffers when an accurate counting of all attack paths is replaced with counting only the shortest paths. In Section IV-C, we provide some experiment results that can reflect the volatility in the attack path elimination outcome caused by enumerating only shortest paths. As we redefine the choke point to be evaluated based on the level to which connections intersect, shortest path mapping-based metrics (*chSP* and *chASP*) remain inadequate to meet the optimization objective. Next, we present a few simple cases to illustrate why.

Firstly, we show with Figure 2 that enumerating shortest paths only, even when all shortest paths are enumerated, cannot collect enough edges for a cut. In this particular

example, it is because s has a minimum cut of 3, and the crucial edge for cutting s is not located in the shortest paths. Even removing all edges in $ASP(s, t)$ cannot cut s from t . Generally, mapping only the shortest paths significantly limits the pool of edges considered as choke points and erratically affects the optimality and accuracy of the result.

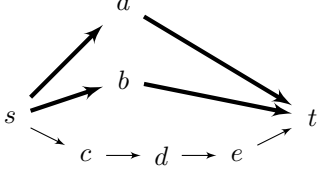


Fig. 2. Break case 1: shortest path edges are highlighted in bold. This example shows that mapping shortest paths limits the pool of edges considered as choke points— s can never be cut from t with only the highlighted edges.

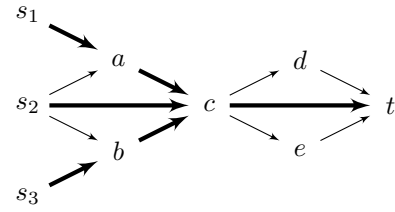
In practice, attack path-based metrics are typically used for producing a ranking list of choke points. The highest-ranking edges are then often considered in greedy order when removing edges to harden security. An example of such implementation can be found in GoodHound [34]. In our second demonstration, we show that the convergence point of the mapped shortest paths at the top of the ranking list is not necessarily optimal for minimizing the number of connected source nodes. In the example given in Figure 3, there are three source nodes. Each source node happens to have one shortest path to t , therefore $chASP$ and $chSP$ would give the same evaluation result. $chSP$ would suggest removing edge (c, t) to disconnect all source nodes, when in fact none of the source nodes are disconnected by removing (c, t) : looking at the full graph, we see that at least three edges should be removed to cut all source nodes.

Suppose now we have a budget of 3. Here, edges in the ranking list of choke points produced by evaluating shortest paths would have been able to cut all source nodes: (a, c) , (s_2, c) and (b, c) form a minimum cut. However, $chSP$ would suggest prioritizing edge (c, t) since it is where all enumerated shortest paths converge, whereas spending one budget on (c, t) would result in at least one source node being impossible to block under the budget.

From this, we see yet another example where edges found on the shortest paths limit our choices for achieving the optimum: (c, t) could have been part of an optimal cut with two more edges $\{(d, t), (e, t)\}$ or $\{(c, d), (c, e)\}$, if those edges were considered in the list.

Furthermore, greedily picking the narrowest edges in the ranking list produced with $chSP$ cannot provide an efficient use of the budget: edges $\{(a, c), (s_2, c), (b, c)\}$ form a minimum cut, yet these edges are not considered the narrowest under $chSP$, and there is no heuristic for biasing the choices towards these less optimal ones.

It could be argued that all attack paths can be eliminated if we dynamically update the graph as we remove one edge at a time. Here, we show that this dynamic approach cannot optimize the use of the defense budget. In Figure 4, the



Edge	$chSP$ / $chASP$
(c, t)	3
(s_1, a)	1
(a, c)	1
(s_2, c)	1
(s_3, b)	1
(b, c)	1

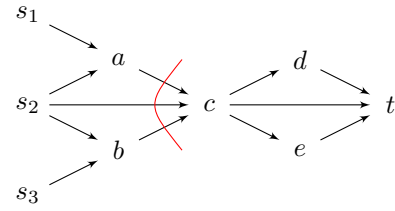


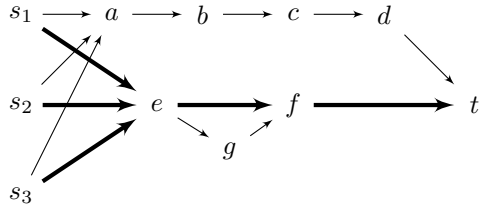
Fig. 3. Break case 2: the location where the shortest paths converge does not indicate the optimal edge for cutting source nodes from t .

minimum cut for disconnecting all source nodes from t is 2. However, 3 edges could potentially be removed if we dynamically update shortest paths as we remove one choke point at a time with $chSP$.

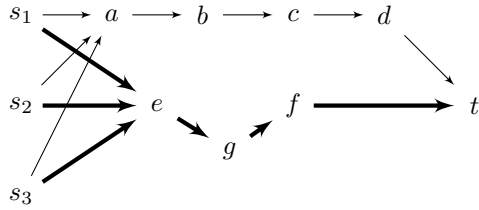
With the above examples, we see that due to highly constricted edge space considered for choke points under $chSP$ and $chASP$, these shortest path-based metrics cannot provide optimal security-hardening results under the connection-based definition of the attack surface and could be misleading even on small simple graphs. When it is under the attack path counting-based definition of the attack surface, the extent to which the optimality of the attack path elimination suffers would be even more challenging to estimate, especially when the graph is generalized to real-world AD attack graphs with considerably larger scales and more complex connectivity.

IV. EVALUATION AND EXPERIMENTS

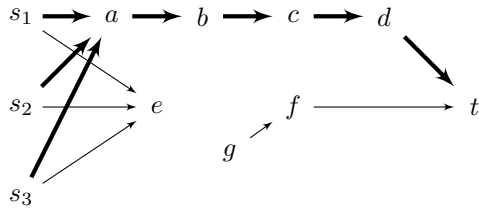
For the evaluation, we first conducted human experiments to observe how the BloodHound shortest path feature is applied in real-world AD attacks. From these observations, we derived key motivations that further justify the new metric over traditional attack path mapping-based metrics. This was followed by a series of experiments comparing the performance of various choke point metrics. The experiments utilized a comprehensive suite of AD datasets, for which we will provide a detailed description.



Edge	<i>chSP</i> / <i>chASP</i>
(e, f)	3
(f, t)	3
(s ₁ , e)	1
(s ₂ , e)	1
(s ₃ , e)	1



Edge	<i>chSP</i> / <i>chASP</i>
(e, g)	3
(g, f)	3
(f, t)	3
(s ₁ , e)	1
(s ₂ , e)	1
(s ₃ , e)	1



Edge	<i>chSP</i> / <i>chASP</i>
(a, b)	3
(b, c)	3
(c, d)	3
(d, t)	3
(s ₁ , a)	1
(s ₂ , a)	1
(s ₃ , a)	1

Fig. 4. Break case 3: dynamic choke point removal does not provide an optimization of the number of edges removed.

A. Human Experiments

We performed a think-aloud study with a group of 8 white-hat hackers, experienced in Active Directory pen-testing. Participants were recruited from those with previous experience using BloodHound for offensive cyber operations and this experience must be hands-on in a job function. We interviewed the participants on how they would attack an AD system. We provided the participants with a synthetic AD attack graph that has 973 nodes and 5155 edges. The total number of low-privileged user accounts and computers connected to DA is 157, which we define to be the source nodes S . The full AD map of the environment in BloodHound was provided to the participants. The participants then used BloodHound to explore the graph and determine their attack strategies—just as in a real-world attack.

Details of the experiment methodology are as follows:

- The familiarization phase: One week before the date organized for the experimentation phase, the participants were sent the required file set to allow them to review the BloodHound map of the simulated network. They are provided with a predefined target to reach (the DomainAdmins security group) and 10 already compromised nodes (9 users and admin rights to 1 computer) to act as initial entry points.
- The think-aloud experiment phase
 - 1) Ask participants to show how they would attack a network on BloodHound. Voice record their thought process. (Present uninformed condition)
 - 2) Repeat step 1, but inform participants about the existence of honeypots. (Present informed condition)
 - 3) Ask participants to reflect on their experiences, including any changes in thought processes and attack patterns, and what they believe the causes to be.

We reviewed the transcripts of the interviews. A summary of the attack strategies suggested by the participants is shown in Table I. There are three recurring techniques the participants would use in exploiting an AD environment: (1) adopting BloodHound's shortest path feature, (2) actively seeking privilege escalation, and (3) frequent pivoting and continuous reconnaissance to explore more options. Looking at the attack graph, all of the experts mentioned using BloodHound to gather information about the environment, and using the shortest path feature to investigate network entities, unusual configurations and areas they are interested in expanding to or can give them a level of privilege escalation, such as more local admin rights.

We observed that even though all participants attempted mapping shortest paths from the initially compromised nodes to DA, none of them committed to those paths. In Table I, "shortest paths" is included as a keyword that appeared in the interviews, however we only marked participants that frequently mentioned shortest paths in their subsequent explorations. 4 experts raised the point that knowing about the presence of decoys makes them more hesitant and slow in their

movement, and also inclined towards more frequent pivoting. 3 experts showed that an informed judgment on the security level and the maturity of the organization affects how they penetrate and pivot in the environment. Once they determined they would not exploit the shortest attack paths they immediately found with BloodHound, they redirected their attention toward other network entities that they described as “low-hanging fruits” and attempted laterally moving to them. Some of the experts used BloodHound’s shortest path feature for this purpose. Meanwhile, they gained more knowledge about the system environment and expanded their portfolio of compromised privileges, therefore building up momentum for subsequent strikes. As a result, the attack path that ultimately leads them to DA from the initially compromised nodes is not necessarily the shortest. Instead, it is often a composite of multiple paths representing different privilege escalation attempts.

Participant	Shortest Paths	Privilege Escalation	Frequent Pivoting
1		✓	✓
2	✓	✓	✓
3		✓	✓
4		✓	✓
5		✓	✓
6	✓	✓	✓
7		✓	✓
8		✓	✓

TABLE I

Results on human experiments: participants that actively engaged in the three key attack techniques.

From the experiment, we see how the attackers’ diverse preferences result in complex pivoting and attack path composition, which is difficult for an uninformed shortest path mapping to capture. Given an AD attack graph in which an accurate assessment of the exploitation risks associated with each network component (be it a node or an edge) is lacking, choke point metrics based on uninformed and approximated attack path counting cannot ensure accurately quantifiable and reliable security-hardening results.

Using the AD attack graph investigated by the experts, we evaluate the performance of *chSP*, *chASP* and *chConn* in reducing the number of nodes in *S* connected to DA when we have a defense budget of 1 and 10. There are 2 nodes among the initially compromised nodes that are not connected to DA to begin with, therefore the actual number of valid entry nodes that the hackers possess at the start is 8. The experiment results are given in Table II, which shows that *chConn* would be significantly more effective in reducing the number of network entities connected with DA. When we allow 10 edges to be removed simultaneously, all of the nodes initially compromised by the hacker are disconnected under *chConn*, and almost all low-privileged network entities are left with no path to DA, therefore it would be nearly impossible for the hacker to find a path to DA despite their effort in pivoting, collecting privileges and expanding their set of compromised entities.

	<i>chSP</i>	<i>chASP</i>	<i>chConn</i>
$b = 1$	0 (0)	0 (0)	18 (0)
$b = 10$	18 (0)	44 (4)	155 (8)

TABLE II

Performance of *chSP*, *chASP* and *chConn* in reducing the number of network entities connected to DA in the AD attack graph adopted for the interview with hackers. The value in the parentheses shows the number of initially compromised nodes that are disconnected.

B. Description of experiment datasets

For our experiments, we first collected snapshots of AD network data from a university. Each AD attack graph is then built with data collected over two weeks. Graph information is provided in the following table.

Graph ID	Nodes	Edges	Source nodes
rw-G1	55949	471160	11021
rw-G2	55878	470779	10970
rw-G3	154330	2611083	69434
rw-G4	154660	2616435	69759
rw-G5	150374	2424225	69626

TABLE III

Information on the real-world AD attack graphs for experiments: the number of nodes, edges and source nodes.

Real-world AD network data is hardly available due to confidentiality reasons. Currently, adopting synthetic AD networks for research experiments is the general practice in academia.

DBCcreator¹ and ADSimulator² are two popular AD simulation tools that many past research papers have adopted for experiments [14], [15], [12], [61], [37], [36]. For DBCcreator, users are allowed to specify the number of nodes to be generated. For ADSimulator, more parameters are given for adjusting the network configuration. Information on our generated AD attack graphs is given in Table IV.

Graph ID	Nodes	Edges	Source nodes
ads-G1	3191	28565	505
ads-G2	6191	57831	1091
ads-G3	9191	92104	1577
ads-G4	12191	123583	2061
dbc-G1	3052	17956	223
dbc-G2	6053	36888	444
dbc-G3	9052	61165	573
dbc-G4	12052	81777	826

TABLE IV

AD attack graphs built with ADSimulator and DBCcreator: the numbers of nodes, edges and source nodes.

ADSynth [37] allows the user to specify the level of node and edge misconfiguration, which determines the security posture of an AD network. For our experiments, we create six AD attack graphs with ADSynth, three of which model a relatively secure system environment, and the others have higher

¹<https://github.com/BloodHoundAD/BloodHound-Tools>

²<https://github.com/nicolas-carolo/adsimulator>

levels of edge misconfigurations. Additionally, we observe that vulnerable AD attack graphs generated with ADSynth have a relatively higher number of low-privileged users and computers connected with DA.

Graph ID	Nodes	Edges	Source nodes
adsynth-vul-G1	1110	6812	549
adsynth-vul-G2	5282	27868	3050
adsynth-vul-G3	10091	58947	6113
adsynth-sec-G1	1047	4761	201
adsynth-sec-G2	5169	25143	170
adsynth-sec-G3	10067	48202	607

TABLE V

AD attack graphs built with ADSynth: the numbers of nodes, edges and source nodes. Graphs with “vul” in the ID are AD attack graphs with higher level of vulnerability. Those with “sec” are the secure ones.

For experiments, we select DA as the target node t , and all low-privileged user accounts and computers that have a path to DA in the graph as the source nodes S .

In addition to the above, we also adopt the dataset on authentication events published by Los Alamos National Laboratory (LANL), which has been frequently used for studying identity-based snowball attacks in the past. LANL dataset includes authentication events that are collected within 58 days from AD servers and computers in their Windows-based network [22], [23], which we can utilize to build authentication graphs. Since credential theft and privilege escalation are part of the vulnerabilities modeled in AD attack graphs, an authentication graph can be considered a subgraph within that broader context.

Each LANL data entry records one authentication event, which has a source user account, destination user account, source computer, destination computer, authentication type, logon type and status specified. We only selected data entries that have valid information recorded and represent successful logons.

For each selected authentication event, we create an edge between the destination user and the destination computer, and similarly between the source user and the source computer. We draw a directed edge from a user to a computer to denote the access privilege. From a computer to a user, we can also draw a directed edge to represent the possibility of credential theft. In this way, a maximum of four edges can be created for each authentication event. Although having two edges going in both directions between a user node and a computer node makes the graph undirected, we keep the edges as two directed edges as they denote different exploitation types. The resulting graph would be a directed bipartite graph.

For our experiments, we build four authentication graphs using four weeks of LANL authentication events, with one week of data for one authentication graph. Considering the dynamic in the network activities including changes in network configuration and clearing of cached credentials, we assume that the authentication graph built with one week of data provides actionable choke point recommendations.

The LANL dataset did not provide privilege levels associated with each user account and computer. Therefore, we select source nodes and target nodes based on the red team events published by LANL, with the source user accounts serving as entry nodes S and destination computers as target nodes. Source user accounts and destination computers that are not present in our graphs are eliminated. To fit into our problem definition, we can convert multiple target nodes into one by adding a super-sink node and creating an edge to connect each target node to the super-sink node. Sizes of the graphs, source nodes and target nodes of the 4 graphs are given in Table VI.

Graph ID	Nodes	Edges	Source nodes	Targets
LANL-w1	34548	508808	94	293
LANL-w2	35253	576190	102	301
LANL-w3	34733	549854	96	297
LANL-w4	35320	557444	96	295

TABLE VI

The number of nodes, edges, source nodes and target nodes in LANL authentication graphs for our experiments.

C. Experimental results

For our first experiment, we pick the edge (choke point) that is evaluated as the narrowest to remove, and we compare the effectiveness of the choke point removal in the elimination of ASP when the narrowness is evaluated based on $chSP$ and $chASP$. Recall that $chSP$ and $chASP$ are two metrics for measuring choke points defined in Section III, and ASP is the set of all shortest paths between source nodes S and the target t . $chASP$ would give the same result as ch in this experiment as we assume ASP to be the full space of paths projected for elimination, whereas $chSP$ evaluates with only one shortest path for each source node. The results are given in Table VII. From this, we see that without an accurate mapping of all paths projected for elimination, the identified choke points cannot provide an optimal elimination of paths.

The same also applies to $chASP$ if we expand the attack paths projected for elimination beyond the shortest paths. For the second experiment, we add a super-source node with a directed edge to each source node, and enumerate 5000 shortest paths from the super-source node to t with Yen’s algorithm [58] and select the last 2000 enumerated paths as those projected for elimination. The last 2000 enumerated are unlikely to be the shortest and we use them to model cases where non-shortest paths affect connectivity. As we assume those 2000 paths to be the full space of paths we target, ch would evaluate the narrowness of edges with the 2000 paths. In this experiment, we also remove the edge that is evaluated as the narrowest (the choke point) according to the metric and compare the attack path elimination results provided by different metrics. As shown in Table VIII, there is a high degree of variability in the relative performance between $chSP$ and $chASP$, and their effectiveness cannot be guaranteed, while choke points identified with accurate path mapping retain their optimality across all problem instances.

Graph ID	<i>chSP</i>	<i>chASP</i>
rw-G1	49.35%	97.89%
rw-G2	49.66%	98.46%
rw-G3	19.96%	99.57%
rw-G4	19.99%	99.70%
rw-G5	49.99%	99.84%
ads-G1	3.53%	12.93%
ads-G2	6.70%	10.09%
ads-G3	2.02%	9.00%
ads-G4	4.50%	6.58%
dbc-G1	6.88%	8.71%
dbc-G2	3.80%	13.09%
dbc-G3	6.33%	8.39%
dbc-G4	7.85%	12.44%
adsynth-vul-G1	26.62%	26.83%
adsynth-vul-G2	3.14%	9.41%
adsynth-vul-G3	2.21%	5.80%
adsynth-sec-G1	65.81%	65.81%
adsynth-sec-G2	82.49%	82.49%
adsynth-sec-G3	27.05%	47.94%

TABLE VII

The percentage of *ASP* eliminated by removing the single narrowest choke point evaluated under *chSP* and *chASP*.

Since both *chSP* and *chASP* would be arbitrary in this problem, even *chASP* cannot be guaranteed to identify choke points that give better performance than *chSP*. Remember from Section III that *SP* denotes the set of shortest paths where there is only one shortest path for each source node. If *SP* happens to allow the choke point to converge at a location that is good for eliminating the 2000 projected paths, having an *ASP* where more paths are included could become an interference and bias the selection of choke point away from the one chosen according to *chSP*.

Graph ID	<i>chSP</i>	<i>chASP</i>	<i>ch</i>
rw-G1	999	1999	1999
rw-G2	1000	2000	2000
rw-G3	188	137	384
rw-G4	211	102	438
rw-G5	1000	2000	2000
ads-G1	99	69	501
ads-G2	144	0	602
ads-G3	77	51	463
ads-G4	23	91	220
dbc-G1	111	135	391
dbc-G2	115	80	302
dbc-G3	128	125	153
dbc-G4	254	36	254
adsynth-vul-G1	168	102	676
adsynth-vul-G2	143	0	230
adsynth-vul-G3	93	0	93
adsynth-sec-G1	63	63	1249
adsynth-sec-G2	257	257	1352
adsynth-sec-G3	93	0	1276

TABLE VIII

The number of projected paths eliminated by choke points evaluated under different metrics. 2000 paths in total are projected for elimination.

In our third experiment, we evaluate *chSP*, *chASP* and *chConn*'s performance in reducing the number of source nodes connected with the target node, and we allow more

than one edge to be removed simultaneously, namely $b > 1$. *chConn* allows global optimization of the set of edges to be removed instead of a greedy approach. However, this global optimization is more challenging to achieve for the path mapping-based metrics since the simple path counting problem is #P-complete. Here, we adopt two methods: the first one uses one ranking list of choke points produced with the static graph, and we greedily pick the top- b choke points from the list to remove. For the second one, we adopt a dynamic approach by iteratively picking the narrowest choke point in the ranking list, removing the edge from the graph and recomputing the ranking list in the updated graph. There would be b iterations in total. The choke point evaluation results are given in Table IX, which show that *chConn* has better performance in cutting source nodes and demonstrates consistent effectiveness, whereas *chSP* and *chASP* do not achieve the same level of optimality, and again with their effectiveness varying erratically across different problem instances. The fact that the set of edges can be globally optimized with *chConn* is also crucial for improving the metric's effectiveness.

Table X provides experiment results with attack graphs built from LANL dataset. As bipartite graphs, LANL graphs have a distinct structure and require a significantly larger budget in order to disconnect a notable amount of source nodes. In our experiments, we set the budget to 100. The same conclusion can be drawn from the results with LANL data: *chConn* provides better guidance for reducing the number of connected source nodes than attack path mapping-based metrics.

Overall from the above experiments, we see that attack path mapping-based metrics not only cannot eliminate projected attack paths with a good level of effectiveness retained among different problem instances due to their high level of approximation and arbitrary nature, they also cannot provide a good source node blocking result in general.

Also, it is noticeable that the efficacy of a choke point varies across different AD datasets, whether in terms of attack path elimination or the reduction of source nodes. Choke points identified in AD attack graphs generated using ADSimulator, DBCreator and ADSynth exhibit lower efficacy compared to those found in the real-world graphs, while those in the LANL networks are even less effective. Among datasets generated with ADSynth, choke points identified in attack graphs that represent higher security levels exhibit greater efficacy. We believe the main reason is the topological characteristics of the AD attack graph, including the density and connectivity distribution. These factors are in turn shaped by factors such as the organizational structure, the security policies, and the severity of misconfiguration in the system.

In a network that exhibits community structure, nodes are distributed among groups where the edges in each group are denser, while connections between groups are sparser [35]. If AD networks with a higher level of security (rw-G1-5 and adsynth-sec-G1-3) topologically resemble the community structure, it would be likely that the low-privileged source nodes and DA are located in different groups, and there is a greater chance that choke points converge at the sparser edges

Graph ID	Source nodes	<i>chSP</i>			<i>chASP</i>			<i>chConn</i>	
		<i>b</i> = 1	<i>b</i> = 10 (static)	<i>b</i> = 10 (dynamic)	<i>b</i> = 1	<i>b</i> = 10 (static)	<i>b</i> = 10 (dynamic)	<i>b</i> = 1	<i>b</i> = 10
rw- <i>G</i> 1	11021	0	10779	11010	<u>10779</u>	11005	11012	<u>10779</u>	11016
rw- <i>G</i> 2	10970	0	10800	10960	<u>10799</u>	10955	10961	<u>10799</u>	10965
rw- <i>G</i> 3	69434	0	69227	0	<u>69223</u>	69223	69317	<u>69223</u>	69423
rw- <i>G</i> 4	69759	0	69558	0	<u>69557</u>	69557	69652	<u>69557</u>	69748
rw- <i>G</i> 5	69626	0	69523	0	<u>69518</u>	69559	69615	<u>69518</u>	69617
ads- <i>G</i> 1	505	2	8	19	0	13	10	<u>4</u>	52
ads- <i>G</i> 2	1091	2	12	12	0	11	9	<u>4</u>	52
ads- <i>G</i> 3	1577	0	4	11	0	12	12	<u>4</u>	50
ads- <i>G</i> 4	2061	3	13	23	0	19	19	<u>6</u>	58
dbc- <i>G</i> 1	223	4	12	20	0	20	16	<u>8</u>	53
dbc- <i>G</i> 2	444	2	17	27	0	5	7	<u>5</u>	60
dbc- <i>G</i> 3	573	2	14	13	0	9	7	<u>5</u>	49
dbc- <i>G</i> 4	826	0	11	21	0	11	8	<u>5</u>	46
adsynth-vul- <i>G</i> 1	549	2	7	27	0	4	27	<u>7</u>	58
adsynth-vul- <i>G</i> 2	3050	0	0	0	0	0	0	<u>7</u>	73
adsynth-vul- <i>G</i> 3	6113	0	0	0	0	0	0	<u>5</u>	190
adsynth-sec- <i>G</i> 1	201	0	9	190	0	7	189	<u>11</u>	191
adsynth-sec- <i>G</i> 2	170	0	139	8	0	99	168	<u>99</u>	169
adsynth-sec- <i>G</i> 3	607	0	4	9	<u>64</u>	69	577	<u>64</u>	581

TABLE IX

The number of source nodes disconnected by choke points evaluated under different metrics. It is desirable to disconnect as many source nodes from *t* as possible. For each graph, optimal results for *b* = 10 are in bold. Optimal results for *b* = 1 are underscored.

Graph ID	Source nodes	<i>chSP</i>			<i>chASP</i>			<i>chConn</i>	
		<i>b</i> = 1	<i>b</i> = 100 (static)	<i>b</i> = 100 (dynamic)	<i>b</i> = 1	<i>b</i> = 100 (static)	<i>b</i> = 100 (dynamic)	<i>b</i> = 1	<i>b</i> = 100
LANL-w1	94	0	2	9	0	0	15	<u>1</u>	18
LANL-w2	102	0	2	8	0	0	19	<u>1</u>	22
LANL-w3	96	0	6	7	<u>1</u>	1	16	<u>1</u>	21
LANL-w4	96	0	5	9	<u>1</u>	2	15	<u>1</u>	21

TABLE X

Results on the disconnection of source nodes with LANL data. Optimal results for *b* = 100 are in bold, and optimal results for *b* = 1 are underscored.

between groups and therefore demonstrate a better efficacy. However, little research has been done on the topological structure of AD attack graphs at the current stage. It is dangerous to assume that metrics based on shortest path mapping would be sufficient merely based on the good results obtained from certain AD datasets. Rigor in the metric definition and implementation remains crucial for guaranteed effectiveness and optimality in security-hardening.

In addition to topological properties inherent in AD networks, the completeness of network data as well as the duration over which network data is collected for constructing the attack graph also play significant roles in influencing its structure and density. The selection of source nodes may also have an impact, since paths leaving different source nodes form different subgraphs, which again have different topological structures. For practical usage, source nodes can be adjusted depending on the use case to identify choke points that prioritize the reinforcement against certain attack entries. The IT administrator can also change the size of choke points to improve the entry coverage.

V. CONCLUSION

In this work, we studied metrics for the identification of attack path choke points in Active Directory attack graphs. The attack graph model we studied is unweighted as the probabilistic parameters quantifying the exploitation risks are difficult to obtain in practice. We strived to mitigate the security risk threatening the safety of the high-value target by identifying attack path choke points to reduce the connectivity this target is exposed to. Attack path choke points are generally described as locations where most attack paths pass through before reaching a high-value target, eliminating which would bring the largest attack path mitigation benefit. Since the problem of counting simple paths in a directed graph is intractable, numerous existing solutions revolve around the mapping and elimination of shortest paths, either as an alternative to the #P-complete simple path counting problem or due to assumptions about optimal attacker's strategies. We believe that replacing comprehensive attack path mapping with shortest path enumeration is a highly simplified approximation for the quantification and visualization of the connectivity. Assumptions about shortest paths invariably being the optimal attacker's strategies also cause various defense blind spots that the attacker can exploit. In a human study we conducted with

white-hat hackers, we observed that even though the attackers adopt BloodHound's shortest path feature, their diverse attack preferences result in complex pivoting and attack path compositions. In light of the above issues, we proposed a novel choke point metric that eliminates the arbitrariness inherent in the shortest path-based choke point metrics, bypasses the intractable simple path counting problem, and redirects the attack path elimination effort into minimizing the number of vulnerable network entities with a path to the high-value target, therefore improving the utility of the choke point metric. The extent to which the optimality of the attack path elimination result suffers due to mere shortest path mapping is challenging to quantify. We provided some experiment results that reflect the volatility in the attack path elimination outcome caused by enumerating only the shortest paths. We also demonstrated break cases where shortest path mapping-based metrics cannot yield optimal results for our new definition of the choke point metric. The analysis is further strengthened with experiments comparing the choke point metrics using a large suite of AD datasets including both real-world AD datasets and attack graphs generated with various AD simulation packages. From the results, we concluded that shortest path mapping-based metrics are indeed unreliable due to their high level of approximation and arbitrary nature. Uninformed attack path or shortest path mapping is incapable of capturing the complex pivoting the attacker performs during an attack and would be counterproductive. Relying on shortest path mapping to substitute the comprehensive attack path mapping would result in great volatility in security-hardening outcomes. Since comprehensive attack path mapping is an insurmountable challenge in the face of the immensity of the attack paths, the connection-based choke point metric would be a better substitute allowing evaluation results with exact optimality. The number of vulnerable network entities disconnected from the high-value target is efficiently and accurately measurable. It also ensures that no alternative paths can be found leaving a potentially compromised node once that node is disconnected from the high-value target, therefore greatly undermining the attacker's pivoting effort. As such, the new metric offers significantly improved optimality and utility in the mitigation of the attack surface.

REFERENCES

- [1] America's Cyber Defense Agency. Ed 21-02: Mitigate microsoft exchange on-premises product vulnerabilities. <https://www.cisa.gov/news-events/directives/ed-21-02-mitigate-microsoft-exchange-premise-s-product-vulnerabilities>. Accessed: 2023-10-14.
- [2] Lewis M Alexander. The role of choke points in the ocean context. *GeoJournal*, 26:503–509, 1992.
- [3] Mridul Sankar Barik, Anirban Sengupta, and Chandan Mazumdar. Attack graph generation and analysis techniques. *Defence science journal*, 66(6):559, 2016.
- [4] Daniel Borbor, Lingyu Wang, Sushil Jajodia, and Anoop Singhal. Optimizing the network diversity to improve the resilience of networks against unknown attacks. *Computer Communications*, 145:96–112, 2019.
- [5] Lucas Bouillot and Emmanuel Gras. Chemins de contrôle en environnement active directory. *Proceedings of the SSTIC conférence 2014*, 2014.
- [6] David Chalupa, Ken A Hawick, and James A Walker. Hybrid bridge-based memetic algorithms for finding bottlenecks in complex networks. *Big Data Research*, 14:68–80, 2018.
- [7] CISA. Russian state-sponsored cyber actors target cleared defense contractor networks to obtain sensitive u.s. defense information and technology. <https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-047a>. Accessed: 2024-09-25.
- [8] Rinku Dewri, Nayot Poolsappasit, Indrajit Ray, and Darrell Whitley. Optimal security hardening using multi-objective optimization on attack tree models of networks. In *Proceedings of the 14th ACM Conference on Computer and Communications Security, CCS '07*, page 204–213, New York, NY, USA, 2007. Association for Computing Machinery.
- [9] John Dunagan, Alice X Zheng, and Daniel R Simon. Heat-ray: combating identity snowball attacks using machinelearning, combinatorial optimization and attack graphs. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 305–320, 2009.
- [10] Karel Durkota, Vilam Lisý, Branislav Bošanský, Christopher Kiekintveld, and Michal Pěchouček. Hardening networks against strategic attackers using attack graph games. *Computers & Security*, 87:101578, 2019.
- [11] Marius Elmiger, Mouad Lemoudden, Nikolaos Pitropakis, and William J Buchanan. Start thinking in graphs: using graphs to address critical attack paths in a microsoft cloud tenant. *International Journal of Information Security*, pages 1–19, 2023.
- [12] Diksha Goel, Max Hector Ward-Graham, Aneta Neumann, Frank Neumann, Hung Nguyen, and Mingyu Guo. Defending active directory by combining neural network based dynamic program and evolutionary diversity optimisation. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '22*, page 1191–1199, New York, NY, USA, 2022. Association for Computing Machinery.
- [13] Michael Greenberg. Why focusing on choke points can help solve your resource constraints. <https://xmcyber.com/blog/why-focusing-on-choke-points-can-help-solve-your-resource-constraints/>. Accessed: 2023-10-12.
- [14] Mingyu Guo, Jialiang Li, Aneta Neumann, Frank Neumann, and Hung Nguyen. Practical fixed-parameter algorithms for defending active directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9360–9367, 2022.
- [15] Mingyu Guo, Max Ward, Aneta Neumann, Frank Neumann, and Hung Nguyen. Scalable edge blocking algorithms for defending active directory style attack graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 5649–5656, 2023.
- [16] Yinghu Huang, Guoyin Wang, and Yuanxiang Tang. Bottleneck attack strategies on complex communication networks. In *Advanced Intelligent Computing Theories and Applications. With Aspects of Artificial Intelligence: 6th International Conference on Intelligent Computing, ICIC 2010, Changsha, China, August 18-21, 2010. Proceedings*, pages 418–425. Springer, 2010.
- [17] Woonchang Hwang, Taehyong Kim, Murali Ramanathan, and Aidong Zhang. Bridging centrality: graph mining from element level to group level. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 336–344, 2008.
- [18] Kent Ickler. Plumhound. <https://github.com/PlumHound>. Accessed: 2024-09-19.
- [19] JD. Bloodhound—calculating ad metrics 0x01. <https://falconforce.nl/bloodhound-calculating-ad-metrics-0x01/>. Accessed: 2023-08-04.
- [20] JD. Bloodhound—calculating ad metrics 0x02. <https://falconforce.nl/bloodhound-calculating-ad-metrics-0x02/>. Accessed: 2023-08-04.
- [21] Somesh Jha, Oleg Sheyner, and Jeannette Wing. Two formal analyses of attack graphs. In *Proceedings 15th IEEE Computer Security Foundations Workshop. CSFW-15*, pages 49–63. IEEE, 2002.
- [22] Alexander D. Kent. Comprehensive, Multi-Source Cyber-Security Events. Los Alamos National Laboratory, 2015.
- [23] Alexander D. Kent. Cybersecurity Data Sources for Dynamic Network Research. In *Dynamic Networks in Cybersecurity*. Imperial College Press, June 2015.
- [24] Justin Kohler. Active directory attack paths — “is it always this bad?”. <https://posts.specterops.io/active-directory-attack-paths-is-everyone-this-bad-44b7538402c#:~:text=Yes%2C%20it's%20this%20bad%20for,%2C%20you're%20not%20alone.&text=While%20the%20answer%20is%20%E2%80%9Cyes,%202B%20years%20of%20misconfiguration%20debt>. Accessed: 2024-05-20.
- [25] Bernhard Korte and Jens Vygen. Network flows. *Combinatorial Optimization: Theory and Algorithms*, pages 173–209, 2012.

- [26] Rajesh Kumar, Enno Ruijters, and Mariëlle Stoelinga. Quantitative attack tree analysis via priced timed automata. In *International Conference on Formal Modeling and Analysis of Timed Systems*, pages 156–171. Springer, 2015.
- [27] Harjinder Singh Lallie, Kurt Debattista, and Jay Bal. A review of attack graph and attack tree visual syntax in cyber security. *Computer Science Review*, 35:100219, 2020.
- [28] Aude Marzuoli, Eric Feron, and Emmanuel Boidot. Resilience of the national airspace system structure: A data-driven network approach. In *2014 IEEE/AIAA 33rd Digital Avionics Systems Conference (DASC)*, pages 1D2–1. IEEE, 2014.
- [29] Microsoft. Attack path analysis and enhanced risk-hunting for containers. <https://learn.microsoft.com/en-us/azure/defender-for-cloud/how-to-test-attack-path-and-security-explorer-with-vulnerable-container-image>. Accessed: 2024-09-25.
- [30] Microsoft. Identify and remediate attack paths. <https://learn.microsoft.com/en-us/azure/defender-for-cloud/how-to-manage-attack-path>. Accessed: 2024-09-25.
- [31] Microsoft. Investigating risk with security explorer/attack paths. <https://learn.microsoft.com/en-us/azure/defender-for-cloud/concept-attack-path>. Accessed: 2024-09-25.
- [32] Microsoft. Overview of attack paths. <https://learn.microsoft.com/en-us/security-exposure-management/work-attack-paths-overview>. Accessed: 2024-09-25.
- [33] Microsoft. Review attack paths. <https://learn.microsoft.com/en-us/security-exposure-management/review-attack-paths>. Accessed: 2024-09-25.
- [34] Andi Morris. Goodhound. <https://github.com/idnahacks/GoodHound>. Accessed: 2022-12-14.
- [35] Mark EJ Newman and Michelle Girvan. Finding and evaluating community structure in networks. *Physical review E*, 69(2):026113, 2004.
- [36] Huy Quang Ngo, Mingyu Guo, and Hung Nguyen. Catch me if you can: Effective honeypot placement in dynamic ad attack graphs. in *Procs. of the IEEE Infocom*, 2024, 2024.
- [37] Nhu Long Nguyen, Nick Falkner, and Hung Nguyen. Adsynth: Synthesizing realistic active directory attack graphs. in *Procs. of The 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 2024, 2024.
- [38] Steven Noel and Sushil Jajodia. Understanding complex network attack graphs through clustered adjacency matrices. In *21st Annual Computer Security Applications Conference (ACSAC'05)*, pages 10–pp. IEEE, 2005.
- [39] Rodolphe Ortalo, Yves Deswarte, and Mohamed Kaâniche. Experimenting with quantitative evaluation tools for monitoring operational security. *IEEE Transactions on Software Engineering*, 25(5):633–650, 1999.
- [40] Gierael Ortega, Alin Nagraj, and Devin Parikh. Center for threat-informed defense, microsoft, and industry partners streamline mitre att&ck® matrix evaluation for defenders. <https://www.microsoft.com/en-us/security/blog/2022/05/11/center-for-threat-informed-defense-microsoft-and-industry-partners-streamline-mitre-attack-matrix-evaluation-for-defenders/>. Accessed: 2023-10-05.
- [41] Xinming Ou, Wayne F. Boyer, and Miles A. McQueen. A scalable approach to attack graph generation. In *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS '06*, page 336–345, New York, NY, USA, 2006. Association for Computing Machinery.
- [42] Xinming Ou, Wayne F. Boyer, and Miles A. McQueen. A scalable approach to attack graph generation. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 336–345, 2006.
- [43] Xinming Ou, Sudhakar Govindavajhala, Andrew W Appel, et al. Mulval: A logic-based network security analyzer. In *USENIX security symposium*, volume 8, pages 113–128. Baltimore, MD, 2005.
- [44] Gopal Pandurangan, Prabhakar Raghavan, and Eli Upfal. Building low-diameter peer-to-peer networks. *IEEE Journal on selected areas in communications*, 21(6):995–1002, 2003.
- [45] Bryan Patton. Understanding attack paths targeting active directory. <https://blog.quest.com/understanding-attack-paths-targeting-active-directory/>. Accessed: 2023-10-12.
- [46] Dekel Paz. Bluehound. <https://zeronetworks.com/blog/bluehound-community-driven-resilience/>. Accessed: 2024-09-19.
- [47] Luke Perkins. Terrain analysis in real-time strategy games: An integrated approach to choke point detection and region decomposition. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 6, pages 168–173, 2010.
- [48] Cynthia Phillips and Laura Painton Swiler. A graph-based system for network-vulnerability analysis. In *Proceedings of the 1998 Workshop on New Security Paradigms, NSPW '98*, page 71–79, New York, NY, USA, 1998. Association for Computing Machinery.
- [49] R.W. Ritchey and P. Ammann. Using model checking to analyze network vulnerabilities. In *Proceeding 2000 IEEE Symposium on Security and Privacy. S&P 2000*, pages 156–165, 2000.
- [50] Andy Robbins. The attack path management manifesto. <https://posts.specterops.io/the-attack-path-management-manifesto-3a3b117f5e5>. Accessed: 2022-12-14.
- [51] Andy Robbins. Why choke-point analysis is essential in active directory security. <https://www.darkreading.com/application-security/why-choke-point-analysis-is-essential-in-active-directory-security>. Accessed: 2023-10-12.
- [52] Andy Robbins, Rohan Vazarkar, and Will Schroeder. Bloodhound: Six degrees of domain admin. <https://bloodhound.readthedocs.io/en/latest/index.html>. Accessed: 2022-12-14.
- [53] Ben Roberts and Dirk P Kroese. Estimating the number of st paths in a graph. *J. Graph Algorithms Appl.*, 11(1):195–214, 2007.
- [54] Reginald E. Sawilla and Xinming Ou. Identifying critical attack assets in dependency attack graphs. In Sushil Jajodia and Javier Lopez, editors, *Computer Security - ESORICS 2008*, pages 18–34, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- [55] Bruce Schneier. Attack trees. *Dr. Dobbs's journal*, 24(12):21–29, 1999.
- [56] Oleg Sheyner, Joshua Haines, Somesh Jha, Richard Lippmann, and Jeannette M Wing. Automated generation and analysis of attack graphs. In *Proceedings 2002 IEEE Symposium on Security and Privacy*, pages 273–284. IEEE, 2002.
- [57] Lifu Wang, Guotao Zhao, Zhi Kong, and Yunkang Zhao. Controllability and optimization of complex networks based on bridges. *Complexity*, 2020(1):6695026, 2020.
- [58] Jin Y Yen. Finding the k shortest loopless paths in a network. *management Science*, 17(11):712–716, 1971.
- [59] Kengo Zenitani. Attack graph analysis: an explanatory guide. *Computers & Security*, page 103081, 2022.
- [60] Kengo Zenitani. A multi-objective cost-benefit optimization algorithm for network hardening. *International Journal of Information Security*, pages 1–20, 2022.
- [61] Yumeng Zhang, Max Ward, Mingyu Guo, and Hung Nguyen. A scalable double oracle algorithm for hardening large active directory systems. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security, ASIA CCS '23*, page 993–1003, New York, NY, USA, 2023. Association for Computing Machinery.
- [62] Yumeng Zhang, Max Ward, and Hung Nguyen. Practical anytime algorithms for judicious partitioning of active directory attack graphs. in *Procs. of IJCAI the 33rd International Joint Conference on Artificial Intelligence*, 2024, 2024.

Algorithm 2 $chSP(G, S, t)$

```

1: edge_counts  $\leftarrow \{\}$ 
2: for each  $s$  in  $S$  do
3:   path  $\leftarrow$  shortest_path( $G, s, t$ )
4:   if path  $\neq \emptyset$  then
5:     for each edge  $(u, v)$  on path do
6:       if  $(u, v)$  not in edge_counts then
7:         edge_counts[ $(u, v)$ ]  $\leftarrow 0$ 
8:       edge_counts[ $(u, v)$ ]  $\leftarrow$  edge_counts[ $(u, v)$ ] + 1
return edge_counts

```

Proposition 1. We use $G^* = (V^*, E^*)$ to denote the directed graph constructed with ASP. G^* is an acyclic graph, and only contains paths in ASP from each $s \in S$ to t . That is, no unintentional paths would be created as we build G^* with nodes and edges in ASP.

Proof. First, we prove that G^* is acyclic. We define

$$\delta(v) = \text{len}(\mathcal{SP}(v, t)).$$

With this, we have

$$\delta(t) = 0$$

$$\delta(u) = \delta(v) + 1, \forall (u, v) \in E^*.$$

Suppose there is a cycle $\langle v_1, v_2, \dots, v_n, v_1 \rangle$ in G^* . This implies that

$$\delta(v_1) = \delta(v_n) + 1 = \delta(v_{n-1}) + 2 = \dots = \delta(v_1) + n,$$

which is contradictory. Therefore, G^* must be acyclic.

G^* also must contain only the shortest paths from each s_i to t , $s_i \in S$. Suppose in addition to a shortest path

$$\mathcal{SP}(s_i, t) = \langle s_i, v_1, v_2, \dots, v_x, v_{x+1}, \dots, v_n, t \rangle,$$

there is a longer path

$$P_{s_i, t} = \langle s_i, v_1, v_2, \dots, v_x, v_y, v_{y+1}, \dots, v_{y+z}, v_{x+1}, \dots, v_n, t \rangle$$

in G^* . The existence of $P_{s_i, t}$ implies that

$$\delta(v_{x+1}) = \delta(v_{y+z}) + 1 = \delta(v_{y+z-1}) + 2 = \dots = \delta(v_x) + z + 1,$$

which contradicts with $\delta(v_{x+1}) = \delta(v_x) + 1$. Therefore, G^* only has paths of shortest lengths from each $s_i \in S$ to t . $\square$

Algorithm 3 DFS algorithm for computing $chConn$ when $b = 1$

```

1: function  $chConn\_DFS(G, S, t)$ 
2:   max_disconnected  $\leftarrow 0$ 
3:   choke_point  $\leftarrow \emptyset$ 
4:   for each edge  $(u, v) \in E$  do
5:     disconnected  $\leftarrow 0$ 
6:      $E \leftarrow E - \{(u, v)\}$ 
7:     reachable_nodes  $\leftarrow$  DFS( $G^T, t$ )  $\triangleright G^T$  is a
      transpose of  $G$ 
8:     for each node  $s \in S$  do
9:       if  $s \notin$  reachable_nodes then
10:        disconnected  $\leftarrow$  disconnected + 1
11:       if disconnected > max_disconnected then
12:        choke_point  $\leftarrow (u, v)$ 
13:        max_disconnected  $\leftarrow$  disconnected
14:      $E \leftarrow E \cup \{(u, v)\}$ 
15:   return choke_point

```
